

Reporte Proof of Concept (POC) de 1C Company

Emiliano Islas y Raúl Mora

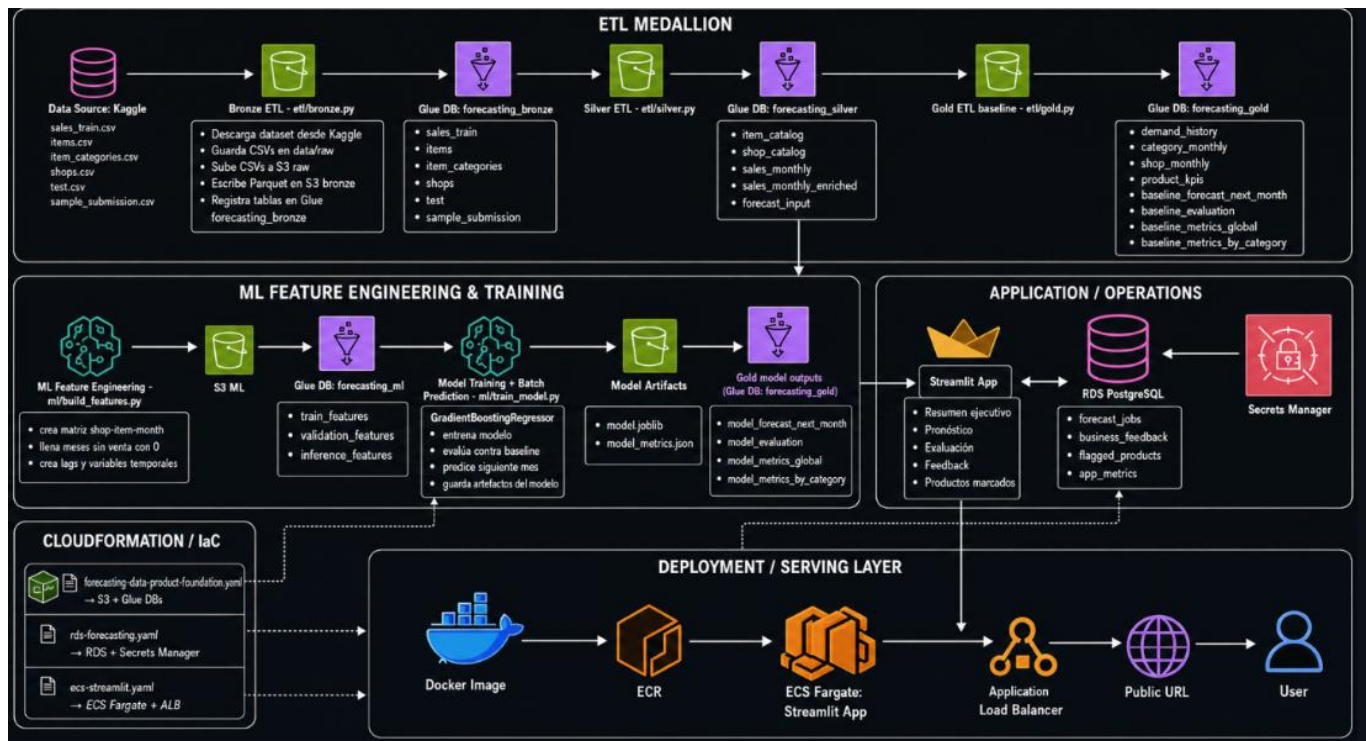
Dentro de 1C company existe una problemática grave con relación al inventario de nuestras tiendas. Esto porque alrededor del 23% de nuestro inventario está en sobrestock, obligándonos a liquidar con descuentos, y también tenemos quiebres de stock en productos clave el 18% del tiempo, generando pérdidas millonarias. Cabe aclarar que contamos con 60 tiendas y 22,170 distintos productos, y nuestra meta es predecir las ventas por producto en cada tienda durante cada mes. Actualmente ajustamos inventarios cada 14 días, mientras que nuestra competencia directa lo hace en 48 horas. Esto puede cambiar con la implementación de un modelo de aprendizaje de maquina para forecasting (predicción) a nivel producto-tienda-mes. Para la correcta implementación de este modelo se cuenta con una base de datos lo suficientemente significativa, pues hay alrededor de 2.9 millones de registros a lo largo de 3 años. A grandes rasgos, el objetivo es desarrollar un modelo de aprendizaje de maquina end to end que prediga ventas futuras en retail para poder maximizar nuestras ganancias.

Nuestra solución como científicos de datos es usar un modelo conocido como “Gradient Boosting”. De esta manera lograremos reducir una métrica muy importante conocida como RMSE, la cual actualmente se tiene con un valor de 11 unidades, lo que representa un turnover de 6.2x, y con la ayuda de este modelo se llega a bajar considerablemente hasta 1 unidad y con un turnover mucho mayor. De manera resumida, estos modelos nos ayudan a comprender comportamientos pasados, para poder predecir el futuro, esto a partir de las bases de datos ya existentes, aunque con algunas modificaciones realizadas a nuestra conveniencia, claro.

Arquitectura de la solución

La arquitectura del sistema es un end-to-end sobre AWS, donde los datos fluyen desde la ingestión hasta el consumo en una aplicación. El proceso inicia con datos crudos provenientes de Kaggle, que se almacenan en Amazon S3 y se organizan bajo una arquitectura Medallion (Bronze, Silver, Gold), permitiendo separar datos sin procesar (bronze), datos limpios (silver) y datos analíticos (gold) . AWS Glue se utiliza como catálogo para registrar las tablas y permitir su consulta mediante Athena. Sobre estas capas se construye el pipeline de Machine Learning, donde se generan features, se entrena un modelo de tipo Gradient Boosting y se producen predicciones en batch, las cuales se almacenan como resultados listos para consumo.

La aplicación Streamlit se despliega en contenedores mediante ECS Fargate, permitiendo acceso vía una URL pública a través de un Application Load Balancer. La app se conecta a una base de datos PostgreSQL en RDS, donde se almacenan el feedback de usuarios, productos marcados y métricas de uso. Las credenciales de acceso se gestionan mediante AWS Secrets Manager. Esta arquitectura permite desacoplar el procesamiento analítico del consumo en la app, mejorando escalabilidad y manteniendo control sobre costos operativos.



Modelo de datos

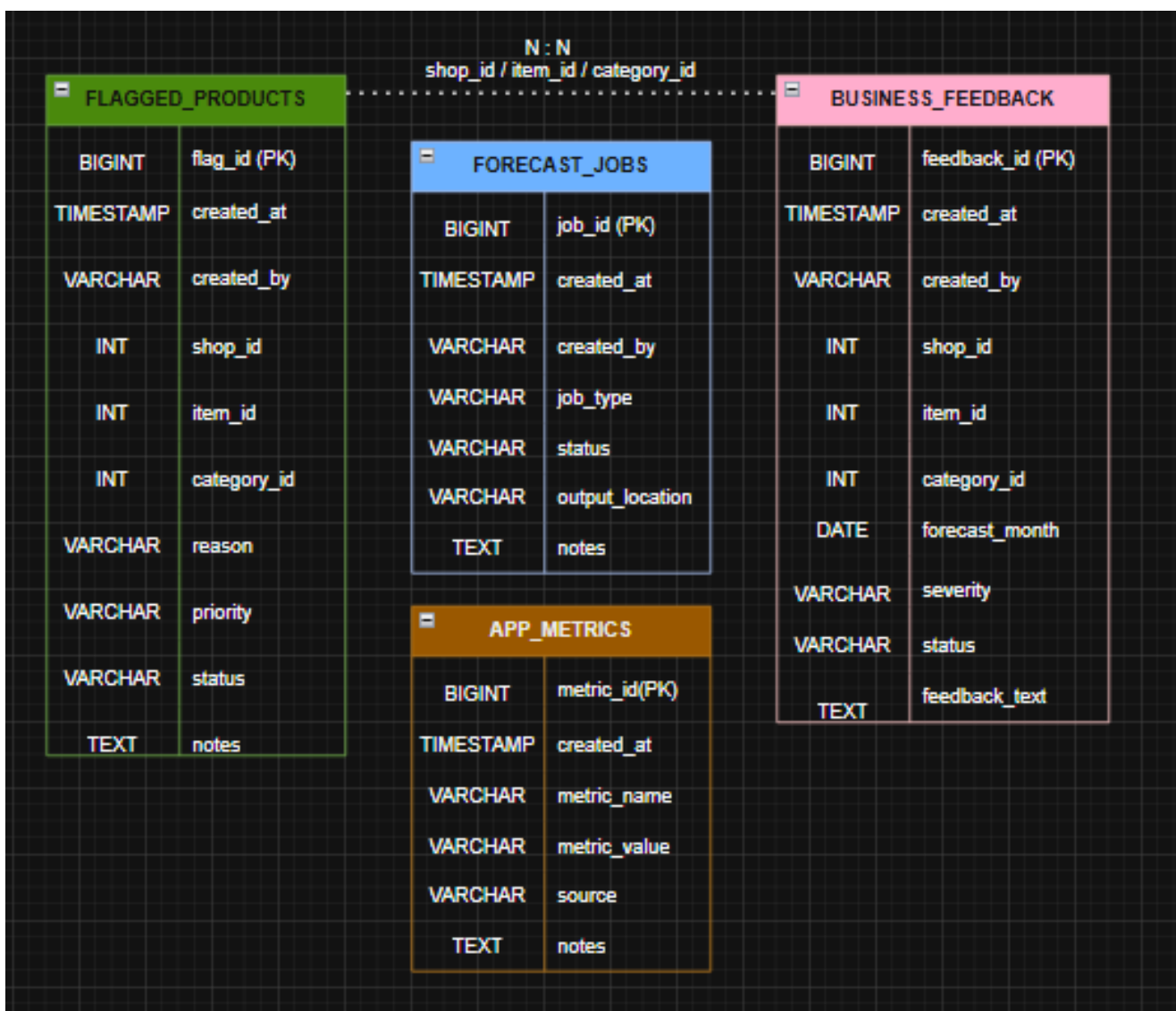
business_feedback : Guarda comentarios del negocio sobre predicciones, categorías, tiendas o productos. Quién escribe: usuarios de la aplicación (equipo de negocio) desde Streamlit. Quién consume: analistas o futuras iteraciones del modelo para mejorar desempeño

flagged_products : Contiene productos marcados como problemáticos para revisión operativa. Quién escribe: usuarios desde la app cuando detectan anomalías. Quién consume: equipo operativo y analítico para monitoreo y debugging del modelo

forecast_jobs : Registra ejecuciones o solicitudes del pipeline de predicción. Quién escribe: scripts de ML / procesos batch. Quién consume: sistema (trazabilidad)

app_metrics : Almacena métricas de uso y eventos de la aplicación. Quién escribe: la aplicación Streamlit. Quién consume: monitoreo del sistema y análisis de uso

Las columnas `shop\_id`, `item\_id` y `category\_id` se usan solo como referencia para identificar la tienda, el producto y la categoría. Estos datos realmente vienen de las tablas analíticas que están en S3/Glue, no de la base en RDS. Por eso en este proyecto no se usan foreign keys reales en PostgreSQL, sino que solo se manejan como referencias para poder conectar la información entre la app y el data lake.



Pipeline de datos y de ML

El pipeline empieza con los datos crudos de Kaggle, que se guardan en S3 y se procesan con una arquitectura Medallion. Primero se ejecuta `etl/bronze.py`, que descarga los archivos, los sube a S3 y registra las tablas en Glue. Después `etl/silver.py` limpia los

datos, arma ventas mensuales, enriquece con catálogos de productos y tiendas, y genera el input para inferencia. Luego `etl/gold.py` construye tablas analíticas, KPIs y el baseline naive. A partir de Silver se ejecuta `ml/build_features.py`, que crea la matriz tienda-producto-mes con variables temporales y lags, y después `ml/train_model.py` entrena el modelo `GradientBoostingRegressor` y genera predicciones batch. El modelo no corre dentro de Streamlit; las predicciones ya quedan precomputadas en tablas Gold y la app solo las consulta. Esto se hizo así porque el input es grande, hay muchas combinaciones tienda-producto, y correr el modelo en cada interacción haría la app más lenta y más cara. De igual forma Streamlit se conecta a RDS para guardar feedback y productos marcados, usando credenciales guardadas en Secrets Manager en lugar de poner usuarios o contraseñas directo en el código (para mayor seguridad).

Evaluación del modelo

La evaluación se hizo usando el último mes histórico como validación. El modelo final obtuvo MAE de 0.346, RMSE de 0.973 y R^2 de 0.267 sobre 238,172 filas de evaluación. La demanda real agregada fue de 61,583 unidades y la demanda predicha fue de 62,792, por lo que el modelo quedó razonablemente cerca a nivel agregado. Además, se compara contra un baseline naive generado en la capa Gold, y la app incluye una sección de evaluación donde se revisa la demanda real contra la predicha por categoría, los grupos con mayor error agregado y los casos individuales con mayor error.

Como el problema tiene muchas combinaciones tienda-producto con ventas cero o muy bajas, la evaluación por categoría y producto ayuda más que ver únicamente el error fila por fila.

Tour de la aplicación

Forecasting MVP

Navegación

- Resumen ejecutivo
- Pronóstico
- Evaluación
- Feedback
- Productos marcados

Gold database: forecasting_gold

RDS configurado

Forecasting Data Product

MVP de planeación de demanda mensual para retail.

¿Qué resuelve este producto?

Este MVP convierte ventas históricas en **pronósticos mensuales de demanda** a nivel **tienda-producto**. La aplicación permite que un usuario de negocio consulte demanda esperada, revise desempeño del modelo y deje feedback operativo desde una URL pública.

Las predicciones se calculan fuera de la app y se guardan en la capa **Gold**. Streamlit solo consulta resultados ya preparados, lo que hace que el dashboard sea rápido y estable.

MAE modelo	RMSE modelo	R² modelo	Filas evaluación
0.346	0.973	0.267	238,172

Demanda real validación	Demanda predicha validación	Diferencia agregada
61,583	62,792	1,209

Modelo final: GradientBoostingRegressor. Feature set: task_01_original_features. Las métricas se calculan usando el último mes histórico como validación.

Hallazgos principales

- La demanda es muy dispersa: muchas combinaciones tienda-producto tienen demanda esperada menor a 1 unidad mensual.
- Por eso, la vista más útil para planeación no es solo la fila granular, sino la demanda agregada por categoría, tienda o producto.
- El modelo permite identificar dónde se concentra la demanda esperada y dónde conviene revisar errores o productos problemáticos.

Resumen ejecutivo

Pronóstico

Evaluación

Feedback

Productos marcados

Gold database: forecasting_gold

RDS configurado

Filtros

Categoría

Accesorios - XB... x

Batteries x

Books - Comics, ... x

Tienda

2 x 3 x 4 x 5 x

6 x

Producto

7877 x 7878 x

7924 x 7925 x

7926 x 7927 x

7928 x 7929 x

7930 x 7931 x

Pronóstico siguiente mes

Predicción mensual de demanda generada por el modelo final.

Las predicciones representan demanda esperada mensual por combinación tienda-producto. En productos de baja rotación es normal observar valores menores a 1. Para planeación, las vistas agregadas por categoría, tienda o producto son más útiles que una fila individual.

Demanda esperada total	Promedio tienda-producto	Combinaciones ≥ 1 unidad	Combinaciones ≥ 0.5 unidad
190.3	0.215	32	54

Por categoría Por tienda Top productos Detalle tienda-producto

Pronóstico agregado por categoría

Top 20 categorías por demanda esperada

Categoría	Demanda esperada
Books - Comics, manga	190.3
Accesorios - XBOX ONE	0.215
Batteries	0.215

Forecasting MVP

Navegación

- ☐ Resumen ejecutivo
- ☐ Pronóstico
- ☒ Evaluación
- ☐ Feedback
- ☐ Productos marcados

Gold database: forecasting_gold

RDS configurado

Evaluación del modelo

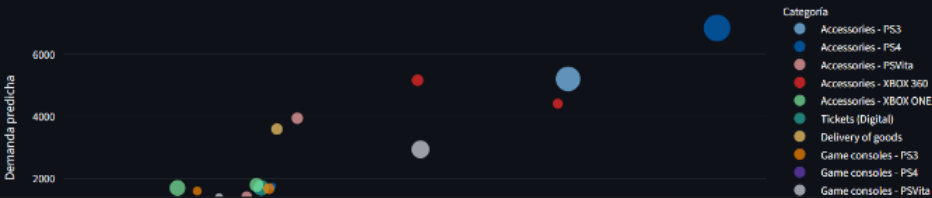
Validación del modelo sobre el último mes histórico disponible.

Esta sección resume qué tan bien el modelo reproduce la demanda observada en el último mes histórico. La vista principal se concentra en el modelo final; la comparación contra baseline queda como referencia técnica.

Filas evaluación	MAE modelo	RMSE modelo	R² modelo
238,172	0.346	0.973	0.267
Demanda real	Demanda predicha		Diferencia agregada
61,583	62,792		1,209
			↑ 2.0%

Demanda real vs demanda predicha por categoría

Demanda real vs predicción del modelo



Forecasting MVP

Navegación

- ☐ Resumen ejecutivo
- ☐ Pronóstico
- ☐ Evaluación
- ☒ Feedback
- ☐ Productos marcados

Gold database: forecasting_gold

RDS configurado

Severidad

medium

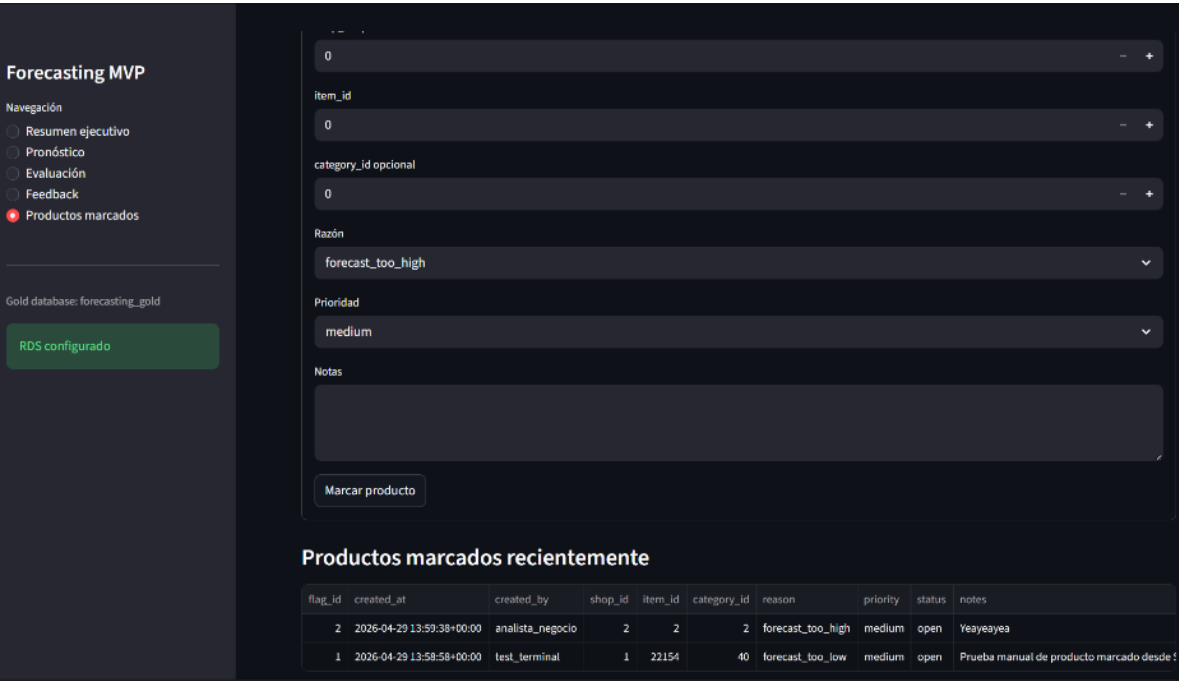
Observación

Ej. La predicción se ve muy baja para esta categoría por temporada alta.

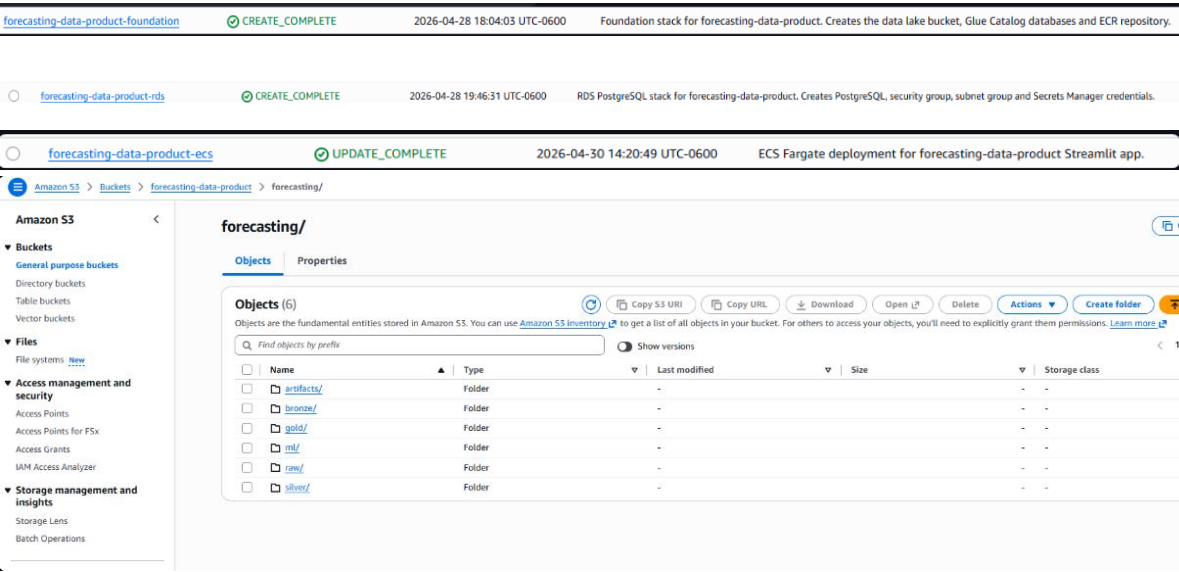
Guardar feedback

Feedback reciente

feedback_id	created_at	created_by	shop_id	item_id	category_id	forecast_month	severity	status	feedback_text
9	2026-04-30 23:57:31+00:00	analista_negocio	None	None	None	34	medium	open	Odio Torreón
8	2026-04-30 19:25:54+00:00	analista_negocio	None	None	None	34	medium	open	andres
7	2026-04-29 20:18:11+00:00	analista_negocio	None	None	None	34	medium	open	esta muy mal
6	2026-04-29 19:19:07+00:00	analista_negocio	None	None	None	34	medium	open	dani
5	2026-04-29 15:58:32+00:00	MORA	2	5	4	34	medium	open	HOLA
4	2026-04-29 15:58:31+00:00	MORA	2	5	4	34	medium	open	HOLA
3	2026-04-29 15:58:19+00:00	analista_negocio	2	5	4	34	medium	open	HOLA
2	2026-04-29 13:58:58+00:00	test_terminal	1	22154	40	34	medium	open	Prueba manual de
1	2026-04-29 13:58:34+00:00	analista_negocio	2	1	2	34	medium	open	feedback



Screenshots de los recursos de AWS desplegados



Amazon ECR > Private registry > Repositories > forecasting-data-product-streamlit > sha256:bff5ab79c167dea150e442c6caf90d1c72634f40d8b65ea479378ee6c3f39cfab

Amazon Elastic Container Registry

Private registry

Public registry

ECR public gallery
Amazon ECS
Amazon EKS

Getting started
Documentation

Image

Details

Image tags

streamlit-clean-20260430234839

URI

780191826160.dkr.ecr.us-east-1.amazonaws.com/forecasting-data-product-streamlit/streamlit-clean-20260430234839

Digest

sha256:bff5ab79c167dea150e442c6caf90d1c72634f40d8b65ea479378ee6c3f39cfab

General information

Artifact type

image

Repository

forecasting-data-product-streamlit

Pushed at

April 30, 2026, 17:49:31 (UTC-06)

Last recorded pull time

April 30, 2026, 17:51:36 (UTC-06)

Size (MiB)

445.65

Image status

Active

Scanning and vulnerabilities

Status

Complete The scan was completed successfully.

Scan completed at

April 30, 2026, 17:49:54 (UTC-06)

Vulnerability source updated at

April 30, 2026, 17:49:54 (UTC-06)

Critical

0

High

2

Medium

4

Low

0

Info

0

Vulnerabilities (6)

CVE-2026-
nhttp2:1.64.0-
HIGH
nhttp2 is an implementation of the Hypertext Transfer Protocol version 2 in C. Prior to version 1.68.1, the nhttp2 library stops reading the incoming data when user facing public API "nhttp2_session_terminate_session" or "nhttp2_session_terminate_session2" is called by the application. They might be called internally by the library when it detects the situation that is subject to connection error. Due to the missing internal state validation, the library keeps reading the rest of the data after one of those APIs is called. Then

Bronze layer for raw 1C forecasting datasets.

Gold layer for analytics-ready forecasting datasets.

Silver layer for cleaned and aggregated forecasting

780191826160

780191826160

780191826160

April 29, 2026 at 00:04:07

April 29, 2026 at 00:04:07

April 29, 2026 at 15:47:11

forecasting_bronze

forecasting_gold

forecasting_ml

forecasting_silver

Amazon Elastic Container Service

Express Mode

Clusters

Namespaces

Task definitions

Daemon task definitions

Account settings

AWS Batch
Amazon EC2
Repositories

Online learning workshop
Documentation
Discover products
Subscriptions

Tell us what you think

forecasting-data-product-cluster

Cluster overview

ARN

arnawsccasus-east-1:780191826160/cluster/forecastin-g-data-product-cluster

Status

Active

CloudWatch monitoring

Default

Registered container instances

-

Services

Draining

Active

1

Tasks

Pending

Running

1

Services (1)

Filter services by value

Any launch type

Any scheduling strategy

Any resource management type

Service name

ARN

Status

Scheduling str...

Launch type

Task definition

Deployments and tasks

Last deployment

forecasting-data-product-streamlit-se...

arnawsccasus-e

Active

Replica

Fargate

forecasting-data-

1/1 Tasks running

Completed

Aurora and RDS > Databases > forecasting-data-product-rds-780191826160

Aurora and RDS

Dashboard

Databases

Performance insights
Snapshots
Exports in Amazon S3
Automated backups
Reserved instances
Proxies

Subnet groups
Parameter groups
Option groups
Custom engine versions
Zero-ETL integrations

Events
Event subscriptions

Recommendations
Certificate update

forecasting-data-product-rds-780191826160

Summary

DB Identifier

forecasting-data-product-rds-780191826160

Status

Available

Role

Instance

Engine

PostgreSQL

Region & AZ

us-east-1b

Connectivity & security

Monitoring

Logs & events

Configuration

Zero-ETL integrations

Maintenance & backups

Data migrations

Tags

Recommendations

Connect using

Code snippets

CloudShell

Endpoints

Internet access gateway

Disabled

Programming language

PSQL (macOS)

Endpoint type

Instance endpoint

Connection steps

Follow the steps below to paste the code of each step in your tool and run the commands. The snippets dynamically reflect the authentication configuration.

1

curl -o global-bundle.pem https://truststore.pki.rds.amazonaws.com/global/global-bundle.pem

2

export RESHOST=forecasting-data-product-rds-780191826160-cylccoy99h-us-east-1-rds.amazonaws.com
psql "host=\$RDSHOST port=5432 dbname=forecasting user=ltan sslmode=verify-full sslrootcert=./global-bundle.pem"

Consideraciones de costo y operación

El componente más costoso es la base de datos en RDS, que cobra por hora mientras está encendida, con un costo aproximado de entre 10 y 20 pesos al día (dependiendo de la configuración que le pongamos). El servicio en ECS Fargate también genera costo mientras la aplicación está corriendo, aunque suele ser menor si solo se usa para demostraciones. Por otro lado, S3 tiene un costo muy bajo por almacenamiento, prácticamente despreciable para este proyecto (centavos).

En un uso continuo, el costo mensual estimado del sistema completo podría estar en un rango de 300 a 800 pesos mexicanos, dependiendo más que nada del tiempo de uso de RDS y ECS.

Para evitar cargos innecesarios, es importante apagar o eliminar los recursos una vez terminada la revisión. Esto incluye eliminar los stacks de CloudFormation asociados a RDS y ECS, detener cualquier servicio activo en ECS, y verificar que no queden instancias corriendo. También se recomienda limpiar imágenes en ECR y revisar archivos grandes en S3.

Limitaciones

Existen diversas limitaciones, como que el modelo predice demanda esperada mensual, no órdenes de inventario directamente. Muchas combinaciones tienda-producto tienen demanda muy baja o cero, por lo que valores menores a 1 son normales. RDS se usa solo para feedback operacional, no como warehouse analítico. No se implementó una réplica de RDS porque la carga esperada del MVP es baja. El modelo se ejecuta batch/offline; la app no recalcula predicciones en tiempo real. No se implementó autenticación de usuarios para la app pública.

Próximos pasos

Agregar autenticación a la app. Automatizar el pipeline con SageMaker Pipelines o Step Functions. Agregar monitoreo de drift de datos y performance del modelo. Incluir intervalos de confianza en predicciones. Agregar lógica de recomendación de inventario basada en reglas de negocio. Incorporar promociones, precios y eventos externos. Agregar CI/CD para build y despliegue automático. Mejorar esquema operacional de RDS con usuarios, auditoría y relaciones normalizadas.

Uso de herramientas de IA

Durante el desarrollo se utilizó asistencia de IA para acelerar: Diseño de arquitectura. Generación inicial de scripts ETL. Debugging de errores de AWS, Docker y Glue. Iteración del dashboard Streamlit.